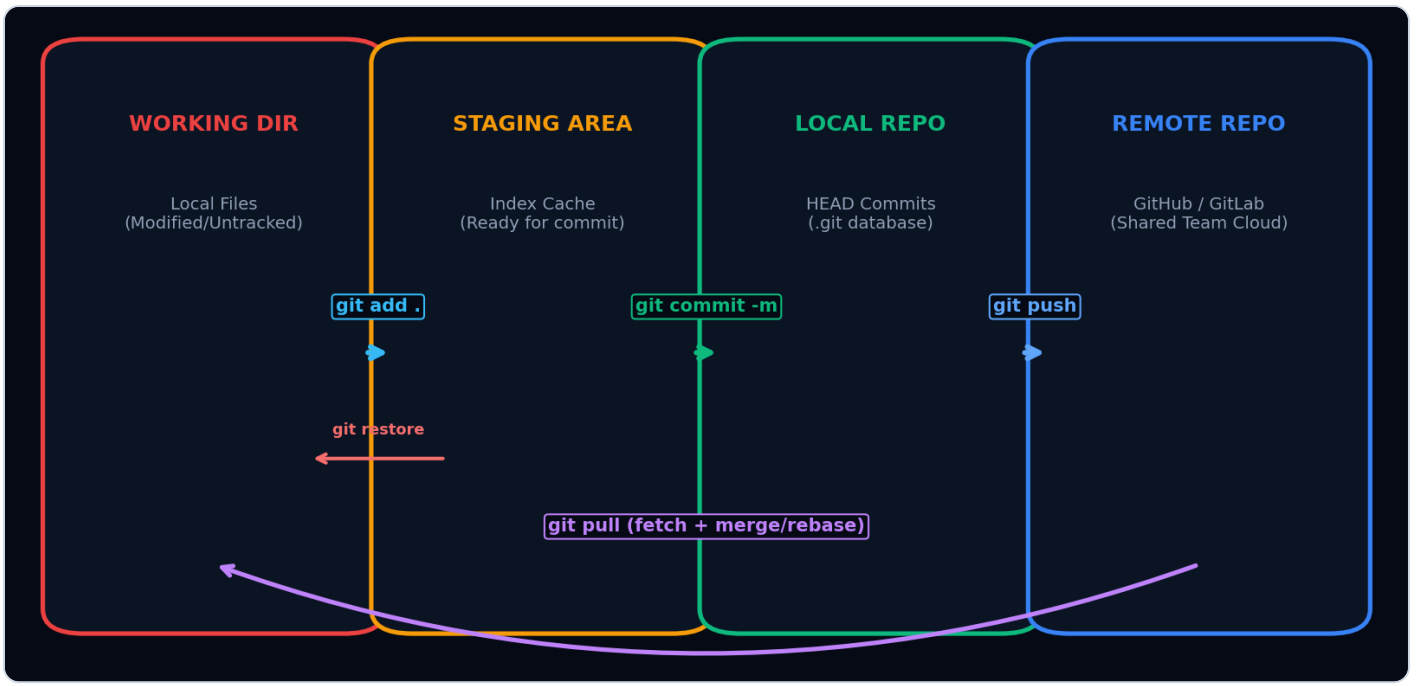
Git & Version Control Cheat Sheet

CORE ARCHITECTURE, STAGING & DAILY OPERATIONS • @SKILL_CIRCUIT

2026 DEVOPS BLUEPRINT



1. CREATE & INITIALIZE

SETUP

Start a new repository locally or clone an existing remote project.

```
# Initialize new local repository in current folder
$ git init
# Clone existing remote repo over secure SSH
$ git clone git@github.com:user/repository.git
# Configure developer identity globally
$ git config --global user.name "Your Name"
$ git config --global user.email "you@email.com"
```

✓ **Default branch:** Ensure default branch is set to `main` with `git config --global init.defaultBranch main`.

2. LOCAL CHANGES & INSPECTION

WORKSPACE

Track modifications in your working tree before staging.

```
# Compact status of modified & untracked files
$ git status -s
# Show unstaged diffs in working directory
$ git diff
# Show diffs already staged and ready to commit
$ git diff --staged
# Stage all tracked + new files
$ git add .
# Interactive chunk-by-chunk staging (Atomic Commits!)
$ git add -p <file>
```

3. COMMITS & REVISIONS

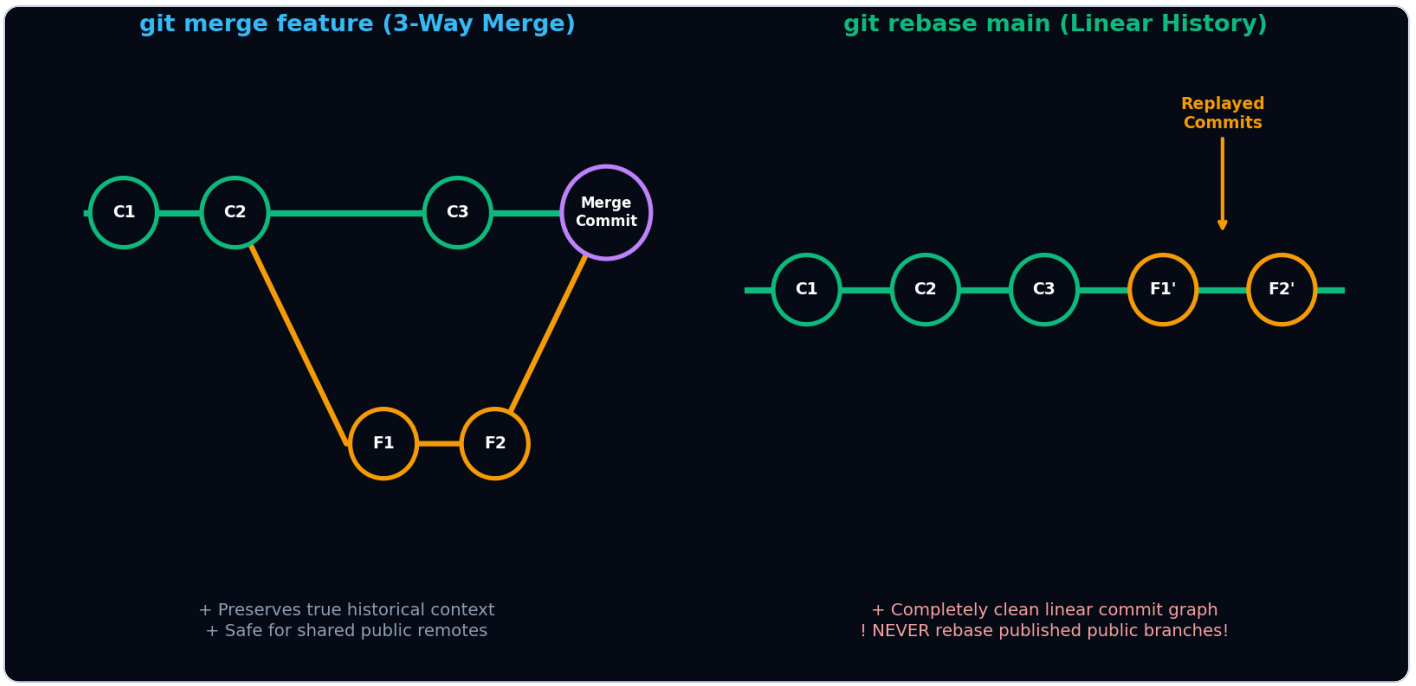
LOCAL HEAD

```
# Commit staged changes with semantic message
$ git commit -m "feat(auth): implement JWT refresh token"
# Stage tracked modified files and commit in one step
$ git commit -a -m "fix: correct pagination boundary"
# Amend last commit message or add forgotten files
$ git commit --amend --no-edit
# Shelf work-in-progress to clean working tree
$ git stash push -m "wip navbar"
$ git stash pop
```

4. COMMIT HISTORY & INSPECTION

LOG & AUDIT

```
# Beautiful colorful ASCII commit DAG graph
$ git log --graph --oneline --decorate --all
# Show exact patch diff for last 2 commits of a file
$ git log -p -2 <file>
# Search commits by keyword in message
$ git log --grep="database migration"
# Inspect who changed what and when in a file
$ git blame -L 10,25 <file>
```



1. BRANCHING & RELEASES — ISOLATION

```
# Create and switch to new branch (Modern command) $ git switch -c feature/payments # Legacy alternative: git checkout -b feature/payments # Switch back to main branch $ git switch main # List all local and remote tracking branches $ git branch -av # Safely delete merged branch (Fails if unmerged!) $ git branch -d feature/payments # Force delete abandoned branch $ git branch -D feature/abandoned # Create immutable production release tag $ git tag -a v1.2.0 -m "Production Release v1.2.0" $ git push origin --tags
```

2. REMOTE SYNC & REBASING — TEAM FLOW

```
# Download latest remote objects without merging $ git fetch origin # Rebase local branch cleanly on updated remote main $ git pull --rebase origin main # Publish local branch to remote repository $ git push -u origin feature/payments # Rebase active branch onto main for linear history $ git rebase main # Resolve rebase conflicts $ git add <resolved-file> $ git rebase --continue $ git rebase --abort # Safe retreat to original state
```

3. THE "OH SH*T, GIT!" EMERGENCY UNDO MATRIX — DISASTER RECOVERY

Mistake Scenario	Recommended Command	Effect on Files	Safety
Discard unstaged typo in file	`git restore <file>`	Restores file to HEAD state	UNSTAGED LOST
Unstage file mistakenly added	`git restore --staged <file>`	Removes from index, keeps edits	100% SAFE
Fix typo in last commit	`git commit --amend -m "new msg"`	Replaces previous commit hash	100% SAFE
Undo last commit (keep work staged)	`git reset --soft HEAD~1`	Rewinds commit; files remain staged	100% SAFE
Undo last commit (unstage all work)	`git reset --mixed HEAD~1`	Rewinds commit; files in editor	SAFE DEFAULT
Nuke commit & all local edits	`git reset --hard HEAD~1`	Permanently wipes working tree	⚠ DESTRUCTIVE
Undo commit already pushed to team	`git revert <commit_hash>`	Creates new inverse commit	✓ SAFE REMOTE
"I lost my code after hard reset!"	`git reflog → git reset --hard HEAD@{n}`	Git time-machine recovers any commit	RESCUES CODE

THE 6 COMMANDMENTS OF VERSION CONTROL (TOWER STANDARD)

CORE HABITS

Principle	Why It Matters	Actionable Best Practice
1. Atomic Commits	Bundling multiple unrelated bugfixes makes rollbacks impossible and code review excruciating.	One logical change per commit. Use `git add -p` to stage individual code hunks.
2. Commit Often	Large commits after days of isolation cause massive merge conflicts and team friction.	Commit frequently in small units. It keeps diffs tiny and makes bisect debugging seamless.
3. No Half-Done Work	Never commit broken, uncompiling code just to go home or clear your branch for a meeting.	Use `git stash push -m "wip"` to safely shelf uncompleted work in your local clipboard.
4. 50/72 Semantic Messages	Vague messages like *"fixed stuff"* destroy repository traceability during production outages.	50-char imperative subject (*feat: add redis cache*), blank line, then explanation of **WHY**.
5. VCS Is NOT a Backup System	Dumping random zip archives or test logs turns repository history into unmaintainable junk.	Commit semantically with proper `.gitignore` rules. Keep repo history architectural and clean.
6. Agree on a Common Workflow	Inconsistent branch naming and conflicting merge strategies lead to deployment errors.	Align the entire engineering team on GitHub-Flow, Git-Flow, or Trunk-Based development.

Top Curated YouTube Video Courses & Channels

100% Free Video Masterclasses

▶ FULL MASTERCLASS

Git & GitHub for Beginners

freeCodeCamp.org (Full Course)

Complete hands-on masterclass covering terminal Git commands, remotes, branches, pull requests, and open source collaboration.

Watch Full Course →

▶ DEVOPS WORKFLOWS

Git Tutorial for DevOps

TechWorld with Nana

Clear, visual explanations of Git branching strategies, Git-flow vs Trunk, merge conflicts, and CI/CD pipeline integration.

Watch on YouTube →

▶ OPEN SOURCE & PRs

Complete Git & GitHub Tutorial

Kunal Kushwaha

Deep dive into commit histories, interactive rebasing, fork workflows, open-source contributing, and GitHub actions.

Watch Tutorial →

▶ PYTHONIC DEV

Git Tutorials for Beginners

Corey Schafer

Clean, methodical walk-through of Git basics, staging, diffing, branching, merging, and remote GitHub synchronization.

Watch Playlist →

▶ VISUAL INTUITION

Git and GitHub is Awesome!

NetworkChuck

High-energy visual explanations of version control concepts, commit hash SHA-1 pointers, branches, and remote repositories.

Watch Video →

▶ OFFICIAL ENTERPRISE

GitHub Universe & Pro Workflows

GitHub Official Channel

Advanced enterprise workflows, security scanning, merge queues, pull request reviews, and GitHub CLI tips.

Watch Official →

WANT MORE DEVOPS & CODING CHEATSHEETS?

Follow **@skill_circuit** on Instagram • Daily cheatsheets, 3D architectural roadmaps & DevOps interview guides!

FOLLOW @SKILL_CIRCUIT